

ความคืบหน้างาน SHAP (Cross-Dataset)

อัปเดตล่าสุด: 28 กันยายน 2569 เวลา 07:00 (เพิ่มข้อ 16 และ 17: รอบ 3 กับ RF, XGBoost, CatBoost)

1. สถานะตอนนี้

ขั้น	สถานะ
เขียนเกณฑ์ล่วงหน้า (pre-registration)	เสร็จ
เขียนสคริปต์	เสร็จ
ทดสอบโค้ดกับข้อมูลชุดเล็ก	ผ่าน
รันจริง รอบ 1	เสร็จ (26 ก.ย. 19:00 ถึง 19:13 ใช้เวลา 12.6 นาที)
ตรวจความทนทาน (เกณฑ์ และโหมด SHAP)	เสร็จ (ข้อ 11, 12)
Part B รอบ 2 พร้อม null	เสร็จ (26 ก.ย. 21:16 ถึง 21:25 ใช้เวลา 8.8 นาที)
รอบ 3 กับ RF, XGBoost, CatBoost	เสร็จ (27 ก.ย. 19:22 ถึง 28 ก.ย. 06:56 รวมเวลารันประมาณ 6.4 ชั่วโมง)
สรุปผล	Part A ไม่ยืนยันตารางเดิม, Part B รอบ 2 และรอบ 3 ไม่สนับสนุน (ดูข้อ 13, 14, 16, 17)

สรุปสั้น

- SHAP ให้ตารางบทบาทของ feature ต่างจาก permutation พอสมควร (κ 0.37 และ 0.33 ต่ำกว่าเกณฑ์ 0.40) SHAP พบ feature ที่นอก dataset ได้ 26 ตัว ส่วน permutation พบ 18 ตัว
- feature ที่เปลี่ยนกลุ่มประมาณครึ่งหนึ่งมี feature อื่นที่สหสัมพันธ์สูง ($|r| \geq 0.9$) ซึ่งตรงกับจุดอ่อนของ permutation ที่คาดไว้
- รอบ 1: คำถามว่า "การทลายพลาตหลังย้าย dataset เกิดจาก fingerprint หรือไม่" ขึ้นกับว่าใช้ตารางไหน (ตาราง SHAP ได้ partial ตาราง permutation ได้ไม่สนับสนุน)
- รอบ 2 (เพิ่ม null จากการสุ่มกลุ่ม feature): ไม่สนับสนุน ทุกกลุ่ม feature ส่วนต่างที่เห็นในรอบ 1 เช่น DDoS +0.139 อยู่ในช่วงที่กลุ่ม feature แบบสุ่มขนาดเท่ากันก็ให้ได้ ($p = 0.14$) จึงยังไม่มีหลักฐานว่า feature ที่นอก dataset เป็นตัวผลักดันให้ทลายพลาตเป็น Benign
- ตรวจความทนทาน: การเปลี่ยนโหมด SHAP แทนไม่มีผล (Spearman 0.98) ส่วนความต่างของ Part A มาจากเกณฑ์ตัดสินส่วนหนึ่ง (ใช้ top-K เดียวกัน κ ขึ้นเป็น 0.40 ถึง 0.62)
- รอบ 3 (RF, XGBoost, CatBoost ด้วยวิธีของรอบ 2): ไม่สนับสนุน ทั้ง 3 โมเดล ทุกกลุ่ม feature ผลของรอบ 2 จึงไม่ได้เกิดกับ LightGBM ตัวเดียว

2. ทำไมต้องทำ SHAP

งานก่อนหน้า (dataset_vs_attack_importance.py) แบ่ง feature 60 ตัวเป็น 4 กลุ่ม คือ fingerprint only, attack only, entangled และ neither โดยใช้ permutation importance

permutation importance มีจุดอ่อนเรื่อง feature ที่ซ้ำกัน ถ้าสอง column มีข้อมูลเดียวกัน สลับค่า column หนึ่งแล้วโมเดลก็ยังอ่านอีก column ได้ ทำให้ทั้งสองตัวได้คะแนนใกล้เคียง 0 ผล adversarial validation บอกแล้วว่าสัญญาณที่นอก dataset กระจายอยู่หลาย column จึงเป็นไปได้ว่าตารางบทบาทของ feature ได้รับผลจากจุดอ่อนนี้

SHAP แบ่งคะแนนให้ column ที่เข้ากันแทนที่จะให้ 0 ทั้งคู่ และอธิบายได้รายแถว งานนี้จึงมี 2 ส่วน

- Part A ตรวจสอบว่าตารางบทบาทของ feature ยังเหมือนเดิมไหมเมื่อใช้ SHAP
- Part B ดูการโจมตีที่โมเดลทายพลาดเป็น Benign หลังย้าย dataset ว่าถูกผลักด้วย feature กลุ่มไหน

3. เกณฑ์ที่ตั้งไว้ก่อนรัน

ไฟล์: results/crossdataset/importance_comparison/SHAP_PREREGISTRATION.md

Part A

- ใช้ข้อมูลและการแบ่งเหมือนงาน permutation ทุกอย่าง (60 features, seed 42-46, 70/30)
- โมเดล LightGBM และ Logistic Regression
- feature ที่ถือว่า "ใช้งาน" ต้องมีส่วนแบ่ง SHAP อย่างน้อย 1/60 และเป็นแบบนี้ใน 4 จาก 5 seeds
- เทียบกับตารางเดิมด้วย Cohen's kappa
- $\text{kappa} \geq 0.60$ ทั้งสองโมเดล = ยืนยันตารางเดิม
- $\text{kappa} < 0.40$ โมเดลใดโมเดลหนึ่ง = ไม่ยืนยัน
- นอกนั้น = partial

Part B

- LightGBM แบบเดียวกับ protocol (77 features, 7 classes, แบ่งตามเวลา) ทศหลักคือ 2017 ไป 2018
- missed = การโจมตีใน test ของอีก dataset ที่ถูกทายเป็น Benign
- caught = การโจมตีคลาสเดียวกันใน test ของ dataset เดิมที่ทายถูก
- วัดว่าแรงที่ผลักไปทาง Benign มาจาก feature กลุ่ม fingerprint only หรือ entangled ก็ส่วน (ค่า s)
- ถ้า $s(\text{missed}) - s(\text{caught}) \geq 0.10$ ทุก seed ในคลาสส่วนใหญ่ = fingerprint ทำให้ทายพลาด
- ถ้าต่างกันไม่เกิน 0.02 ในคลาสส่วนใหญ่ = ไม่สนับสนุน
- นอกนั้น = partial

4. สิ่งที่ทำไปแล้ว

1. ติดตั้ง shap 0.52.0 ใน .venv (มีใน requirements.txt แต่ยังไม่ได้อัปเดต)
2. เขียน scripts/shap_importance.py ใช้ฟังก์ชันเดิมจากสคริปต์ของเพื่อน (dataset_vs_attack_importance.py, adversarial_validation.py, run_crossdataset.py) เพื่อให้ข้อมูลการแบ่ง และโมเดลเหมือนกันทุกอย่าง
3. ทดสอบโค้ดกับข้อมูลชุดเล็ก (1 seed, 20,000 แถวต่อ dataset) ใช้ไฟล์ตัวอย่าง 2018 ที่ทำความเข้าใจเฉพาะสำหรับทดสอบ ตัวเลขจากการทดสอบนี้ ไม่ใช่ผลการทดลองใช้ดูแค่ว่าได้ทำงานถูก

5. สิ่งที่เจอระหว่างทดสอบ และสิ่งที่แก้

ทุกข้อแก้ก่อนรันจริง และบันทึกไว้ในไฟล์เกณฑ์แล้ว

เรื่องที่เจอ	ที่แก้
SHAP แบบ interventional ของ LightGBM ใช้เวลา 9 นาทีต่อ 1 ครั้ง ถ้ารันครบจะใช้ประมาณ 20 ชั่วโมง	LightGBM เปลี่ยนเป็นแบบ tree_path_dependent ทดสอบทั้งหมดเหลือ 1.4 นาที
shap ลด background จาก 200 เหลือ 100 แถวเองโดยไม่บอก	กำหนด max_samples ให้ใช้ครบ 200 (Logistic Regression)

เรื่องที่เจอ	ที่แก้
Part B เอาคลาส Infiltration มาตัดสินทั้งที่ฝั่ง caught มีแค่ 10 แถว	เพิ่มเกณฑ์ caught อย่างน้อย 30 แถว (ค่าเดียวกับ protocol)
เกณฑ์ฝั่ง attack ยังไม่ได้บอกกว่านับ dataset ไหน	ใช้ "dataset ใดก็ได้" เหมือนงาน permutation
จำกัดแถวใน Part B ที่ 2,000 ต่อกลุ่ม ทำให้คลาสเล็กแทบไม่เหลือ	เปลี่ยนเป็น 1,000 แถวต่อคลาส

เรื่องการเปลี่ยนเป็น tree_path_dependent: ตอนแรกตั้งใจใช้ interventional ทั้งสองโมเดล เพื่อให้วัดแบบเดียวกัน แต่ kappa ของแต่ละโมเดลเทียบกับตาราง permutation ของโมเดลตัวเอง การที่สองโมเดลใช้คนละแบบจึงไม่กระทบการตัดสินของ Part A

6. การรันจริง

- ข้อมูล 2018 เต็มชุดได้จากเพื่อน (cicids2018_clean.parquet 11,445,955 แถว สร้างจาก commit cb5361c วันที่ 9 ก.ย.) วางไว้ที่ data/ids2018/
- สุ่มมา dataset ละ 300,000 แถว, 60 features, seed 42-46
- ก่อนรัน สคริปต์บันทึก sha256 ของไฟล์เกณฑ์ไว้ใน summary.json (8f701b15...f85173) ถ้ามีการแก้เกณฑ์หลังเห็นผลจะตรวจได้
- ผลอยู่ที่ results/crossdataset/importance_comparison/shap/

7. ผล Part A: ตารางบทบาทของ feature

ค่าตัดสินตามเกณฑ์: ไม่ยืนยัน (kappa ต่ำกว่า 0.40 ทั้งสองโมเดล)

	LightGBM	Logistic Regression
Cohen's kappa	0.366	0.334
Spearman ของอันดับ attack importance (2017 / 2018)	0.68 / 0.68	0.74 / 0.88
feature ที่เปลี่ยนกลุ่ม	25 จาก 60	29 จาก 60
ในนั้นมีคู่ที่สหสัมพันธ์ ≥ 0.9	13	16
ในนั้นเป็นตัวแทนกลุ่ม feature ซ้ำ	7	6

จำนวน feature ในแต่ละกลุ่ม

กลุ่ม	LightGBM (perm / SHAP)	LR (perm / SHAP)
fingerprint only	10 / 13	2 / 12
entangled	8 / 13	16 / 14
attack only	9 / 5	33 / 18
neither	33 / 29	9 / 16

สิ่งที่เห็น

- ฝั่ง dataset SHAP พบ feature ที่นอก dataset ได้ 26 ตัว ส่วน permutation พบ 18 ตัว ตรงกัน 13 ตัว SHAP พบเพิ่ม 13 ตัว และไม่นับ 5 ตัวที่ permutation นับ
- LightGBM มี feature ที่ permutation ให้เป็น neither ย้ายไป fingerprint only 6 ตัว และ entangled 3 ตัว ตรงกันที่ทำนายไว้ในเกณฑ์ ส่วนกลุ่ม entangled เดิม 7 จาก 8 ตัวยังอยู่ที่เดิม

- LR ต่างมากเพราะเกณฑ์ 1/60 เข้มกว่าเกณฑ์ noise floor ของ permutation สำหรับ LR feature ที่นับว่ามีสัญญาณการโจมตีลดลงจาก 39 / 33 ตัว เหลือ 25 / 16 ตัว (2017 / 2018) LR กระจายน้ำหนักไปหลาย feature ส่วนแบ่งแต่ละตัวจึงต่ำกว่า 1/60 ได้ง่าย ความต่างของ LR จึงมาจากเกณฑ์ตัดสินส่วนหนึ่ง ไม่ได้มาจากวิธี SHAP อย่างเดียว
- feature ที่เปลี่ยนกลุ่มประมาณครึ่งหนึ่งมีคู่ที่สหสัมพันธ์สูง เช่น Packet Length Mean กับ Average Packet Size, Fwd IAT Max กับ Idle Max ตรงกับเหตุผลที่ทำให้ SHAP ดังแต่แรก
- หมายเหตุ: การตรวจสอบสหสัมพันธ์ใช้ข้อมูล 2017 ครบ 60 features แต่ไฟล์ 2018 ตรงชื่อแค่ 24 ตัว (ชื่อ column ต่างกัน) ผลส่วนนี้จึงอิง 2017 เป็นหลัก

8. ผล Part B: การโจมตีที่ถูกทายพลาดหลังย้าย dataset

ทิศหลัก 2017 ไป 2018 ค่าเฉลี่ย 5 seeds (s = ส่วนของแรงที่ผลักไปทาง Benign ที่มาจาก feature กลุ่ม fingerprint only และ entangled)

คลาส	ถูกตัดสินใหม่	s(missed) - s(caught) ตาราง SHAP	ตาราง permutation
DDoS	ใช่	+0.139 (ทุก seed >= 0.10)	-0.076
DoS	ใช่	+0.135 (4 จาก 5 seeds >= 0.10)	+0.054
Web Attack	ใช่	-0.054	-0.098
Infiltration	ไม่ (caught แค่ 10 แถว)	+0.224	0.000
Bot	ไม่ (missed แค่ 8 แถว)	+0.050	-0.065
Brute Force	ไม่ (caught 0 แถว)	-	-

ค่าตัดสินตามเกณฑ์

- ใช้ตาราง SHAP: partial (สนับสนุน 1 คลาส, ไม่สนับสนุน 1 คลาส, กลาง ๆ 1 คลาส)
- ใช้ตาราง permutation: ไม่สนับสนุน
- เนื่องจาก Part A ไม่ยืนยัน ต้องรายงานทั้งสองแบบ ตามที่เขียนไว้ในเกณฑ์

สิ่งที่เห็น

- คำตอบของ Part B ขึ้นกับว่าใช้ตารางไหนนิยาม feature fingerprint ตอนนี้ก็ยังพูดไม่ได้ว่า fingerprint เป็นสาเหตุที่ทำให้ทายพลาด
- กับ DDoS ผลชัดที่สุด ถ้าใช้ตาราง SHAP ทุก seed ได้ส่วนต่างเกิน 0.10 (0.113 ถึง 0.165)
- Web Attack ไปทางตรงข้ามในทั้งสองตาราง
- Brute Force ของ 2017 ในการแบ่งตามเวลาได้ F1 = 0 อยู่แล้วใน protocol_v2 จึงไม่มีแถวที่ทายถูกให้เทียบ ไม่ใช่ว่าปัญหาของสคริปต์นี้
- ค่า s ของตาราง SHAP สูงกว่า (0.6 ถึง 0.8) เพราะกลุ่ม F มี 26 ตัว ส่วนตาราง permutation มี 18 ตัว การเทียบจึงดูที่ส่วนต่างภายในตารางเดียวกันเท่านั้น

ทิศ 2018 ไป 2017 (รายงานไว้ แต่ไม่ใช่ตัดสิน) ส่วนต่างเป็นบวกเกือบทุกคลาส เช่น DDoS +0.341 (ตาราง SHAP) และ +0.197 (ตาราง permutation), Web Attack +0.218 และ +0.178 ทิศนี้ย้าย dataset ได้ดีกว่าอยู่แล้ว ผลนี้น่าสนใจแต่ไม่ได้อยู่ในเกณฑ์ตัดสิน

9. ข้อจำกัด

- LightGBM ใช้ SHAP แบบ path-dependent ส่วน LR ใช้แบบ interventional (เหตุผลอยู่ในข้อ 5)

- เกณฑ์ 1/60 ไม่ได้เหมาะกับทุกโมเดลเท่ากัน โมเดลที่กระจายน้ำหนัก เช่น LR จะมี feature ผ่านเกณฑ์น้อยกว่า
- Part B ตัดสินได้แค่ 3 คลาส จาก 6 คลาสการโจมตี
- ใช้แค่ LightGBM และ LR ยังไม่ได้ทำ RF, XGBoost, CatBoost

10. ขั้นต่อไปที่เป็นไปได้

1. ให้เพื่อนตรวจผลและเกณฑ์ ก่อนนำไปเขียนในรายงานหลัก
2. ขยาย Part A และ B ไป RF, XGBoost, CatBoost ด้วยเกณฑ์เดิม (TreeExplainer เร็ว)
3. ดู DDoS เป็นกรณีศึกษา ว่า feature ไหนผลักไปทาง Benign มากที่สุด
4. นำผลไปใช้กับงาน binary และ distribution alignment ที่ยังค้างอยู่

11. ตรวจสอบความทนทาน 1a: เปลี่ยนเกณฑ์ตัดสิน

เป็นการวิเคราะห์เพิ่มเติม ไม่ได้เปลี่ยนค่าตัดสินของ Part A สคริปต์ `scripts/shap_threshold_check.py` ใช้ผลที่บันทึกไว้ ไม่ได้เทรนใหม่

Part A รอบ 1 เปลี่ยนสองอย่างพร้อมกัน คือวิธีวัด (SHAP แทน permutation) และเกณฑ์ (1/60 แทน noise floor) ข้อนี้ใช้เกณฑ์ top-K แบบเดียวกันกับทั้งสองวิธี เพื่อดูผลของวิธีวัดอย่างเดียว

K	ฝั่ง dataset ตรงกัน	kappa LightGBM	kappa LR
5	80%	0.524	0.486
10	77%	0.448	0.411
15	83%	0.517	0.622
20	80%	0.403	0.481

เมื่อใช้เกณฑ์เดียวกัน kappa ขึ้นจาก 0.33 ถึง 0.37 เป็น 0.40 ถึง 0.62 ความต่างในรอบ 1 จึงมาจากเกณฑ์ส่วนหนึ่ง แต่สองวิธีก็ยังไม่ตรงกันทั้งหมด (ส่วนใหญ่อยู่ในช่วง "moderate")

12. ตรวจสอบความทนทาน 1b: เปลี่ยนโหมด SHAP

สคริปต์ `scripts/shap_mode_check.py` ใช้ LightGBM ของ Part A หนึ่งตัว (2017, seed 42) อธิบาย 300 แถวเดียวกันทั้งสองแบบ (background 100 แถว)

	ค่า
Spearman ของส่วนแบ่ง feature	0.98
top-10 feature ตรงกัน	10 จาก 10
ผ่านเกณฑ์ 1/60	15 ตัวทั้งสองแบบ ตัดสินตรงกัน 96.7%
เวลา	path-dependent 3 วินาที, interventional 144 วินาที

การเปลี่ยนเป็น path-dependent ในรอบ 1 จึงแทบไม่กระทบผล ข้อจำกัด: ตรวจแค่ 1 โมเดล 1 seed และฝั่ง 2017 เท่านั้น

13. Part B รอบ 2: เพิ่ม null จากการสุ่มกลุ่ม feature

เกณฑ์: results/crossdataset/importance_comparison/SHAP_ROUND2_PREREGISTRATION.md
เขียนหลังเห็นผลรอบ 1 และระบุไว้ในไฟล์ ผลรอบ 1 ยังเป็นค่าตัดสินเดิม ไม่ได้ถูกแทนที่ สคริปต์
scripts/shap_round2.py ผลอยู่ที่ results/crossdataset/importance_comparison/shap_round2/

สิ่งที่เปลี่ยนจากรอบ 1

- ใช้กลุ่ม F แบบ consensus คือ feature ที่ทั้งสองวิธีเห็นตรงกันว่านอก dataset ได้ (13 ตัว)
ไม่ต้องเลือกว่าจะเชื่อตารางไหน
- สุ่มกลุ่ม feature ขนาดเท่ากัน 2,000 ครั้ง แล้วดูว่าส่วนต่าง G ของ F จริงสูงกว่ากลุ่มสุ่มกี่เปอร์เซ็นต์ (ค่า p)
- class ที่สนับสนุนต้องได้ $p < 0.05 / 3$ (Bonferroni)

โมเดล แกว และการตั้งค่า SHAP เหมือนรอบ 1 ทุกอย่าง ค่า G ของตาราง SHAP และ permutation
ในรอบนี้ตรงกับรอบ 1 ทุกคลาส (เช่น DDoS +0.1385) ยืนยันว่ารันซ้ำได้ผลเดิม

ทิศหลัก 2017 ไป 2018

คลาส	กลุ่ม F	G	95% ของกลุ่มสุ่ม	p	ผล
DDoS	consensus (13)	-0.021	0.179	0.57	ไม่สนับสนุน
DoS	consensus (13)	+0.066	0.209	0.32	ไม่สนับสนุน
Web Attack	consensus (13)	-0.012	0.243	0.59	ไม่สนับสนุน
DDoS	SHAP (26)	+0.139	0.218	0.14	ไม่สนับสนุน
DoS	SHAP (26)	+0.135	0.238	0.19	ไม่สนับสนุน
Web Attack	SHAP (26)	-0.054	0.330	0.60	ไม่สนับสนุน
DDoS	perm (18)	-0.076	0.203	0.71	ไม่สนับสนุน
DoS	perm (18)	+0.054	0.225	0.34	ไม่สนับสนุน
Web Attack	perm (18)	-0.098	0.283	0.71	ไม่สนับสนุน

ค่าตัดสินตามเกณฑ์: ไม่สนับสนุน (กลุ่ม consensus ซึ่งเป็นกลุ่มหลัก และอีกสองกลุ่มก็ได้ผลเดียวกัน)

สิ่งที่เห็น

- ส่วนต่าง +0.139 ของ DDoS ที่ดูชัดในรอบ 1 อยู่ต่ำกว่าช่วงบนของกลุ่มสุ่ม (0.218) กลุ่ม feature
สุ่มขนาดเท่ากันให้ส่วนต่างแบบนี้ได้บ่อย ในรอบ 1 จึงอ่านผลนี้มากเกินไป
- ช่วงของกลุ่มสุ่มกว้าง (ช่วงบนประมาณ 0.18 ถึง 0.33)
แปลว่าแกวที่หายพลาดต่างจากแกวที่หายถูกในหลาย feature ไม่ได้กระจุกอยู่ที่ feature ที่นอก dataset
- ข้อสรุป: จาก LightGBM และมุมมองของ SHAP ยังไม่มีหลักฐานว่า feature fingerprint
เป็นตัวผลการโจมตีให้ถูกทายเป็น Benign หลังย้าย dataset ต้องหาคำอธิบายอื่น เช่น การกระจายของค่า
feature ที่เลื่อนไปทั้งหมด ซึ่งตรงกับแนวทาง distribution alignment ของอาจารย์

ทิศ 2018 ไป 2017 (ไม่ใช่ตัดสิน) มีบางค่าที่ p ต่ำ เช่น DDoS กับตาราง SHAP $p = 0.0015$ และ Web Attack
กับ consensus $p = 0.049$ แต่เป็นทิศที่ไม่ได้ตั้งเป็นคำถามหลัก และมีการทดสอบหลายครั้ง (18 ค่าต่อทิศ)
จึงถือเป็นข้อสังเกตที่ต้องตรวจเพิ่ม ไม่ใช่ข้อค้นพบ

14. สรุปทั้งหมด

1. SHAP กับ permutation ไม่ตรงกันพอที่จะยืนยันตารางบทบาทของ feature (Part A)
ส่วนหนึ่งเพราะเกณฑ์ ส่วนหนึ่งเพราะ feature ที่ซ้ำและสัมพันธ์กัน

2. ไม่มีหลักฐานว่า feature ที่นอก dataset เป็นสาเหตุที่การโจมตีถูกทายพลาดหลังย้าย dataset (Part B รอบ 1 ขึ้นกับตาราง รอบ 2 ไม่สนับสนุนทุกกลุ่ม)
3. ผลนี้ชี้ว่าการตัด feature fingerprint ทิ้งอย่างเดียวไม่น่าจะแก้ปัญหการย้าย dataset ได้ ซึ่งสอดคล้องกับผล ablation เดิมที่ตัด feature แล้วไม่ดีขึ้น

15. เกณฑ์ที่ต้องผ่านแบบละเอียด (null 200,000 ครั้ง)

เป็นข้อมูลเสริม ไม่ได้เปลี่ยนค่าตัดสินของรอบ 2 (ค่าตัดสินใช้การสุ่ม 2,000 ครั้ง seed 2026 ตามเกณฑ์) สคริปต์ scripts/shap_null_thresholds.py ผลอยู่ที่ shap_round2/null_thresholds.csv

ขั้นตอน

- รันรอบ 2 ซ้ำ แล้วบันทึกค่า SHAP รายแถวไว้ (shap_round2/blocks.joblib) ผลรันซ้ำตรงกับครั้งแรก ต่างกันไม่เกิน 0.0001 ใน 2 ถึง 3 ค่า (มาจากการคำนวณหลาย thread ของ LightGBM) ค่า p และค่าตัดสินเหมือนเดิมทุกค่า
- ระหว่างรันซ้ำ RAM ไม่พอตอนโหลดข้อมูล 2018 สองครั้ง จึงเปลี่ยนวิธีอ่านไฟล์ parquet ให้ใช้ RAM น้อยลง ค่าข้อมูลเหมือนเดิม (จำนวนแถวและจำนวนแต่ละคลาสตรงกับรอบแรกทุกค่า)
- สุ่มกลุ่ม feature เพิ่มทีละขั้น 2k, 5k, 10k, 20k, 50k, 100k, 200k (seed 2027 แยกจากตัวที่ใช้ตัดสิน) หยุดเมื่อจุด 98.33% เปลี่ยนน้อยกว่า 0.002 ซึ่งถึงที่ 200,000 ครั้ง (เปลี่ยน 0.0013)

จุด 98.33% คือ $1 - 0.05/3$ เป็นค่าที่ G ต้องเกินถึงจะผ่านเกณฑ์ Bonferroni

ทิศหลัก 2017 ไป 2018

กลุ่ม F	คลาส	G ที่ได้	ต้องได้ (98.33%)	ช่วงความเชื่อมั่น 95%	ขาดอีก	p (200k)
consensus (13)	DDoS	-0.021	0.233	0.232 ถึง 0.234	0.254	0.58
consensus (13)	DoS	+0.066	0.257	0.256 ถึง 0.259	0.191	0.31
consensus (13)	Web Attack	-0.012	0.293	0.292 ถึง 0.294	0.305	0.59
SHAP (26)	DDoS	+0.139	0.278	0.276 ถึง 0.280	0.139	0.16
SHAP (26)	DoS	+0.135	0.313	0.311 ถึง 0.315	0.178	0.20
SHAP (26)	Web Attack	-0.054	0.395	0.393 ถึง 0.397	0.449	0.60
perm (18)	DDoS	-0.076	0.256	0.255 ถึง 0.258	0.332	0.73
perm (18)	DoS	+0.054	0.289	0.287 ถึง 0.290	0.234	0.36
perm (18)	Web Attack	-0.098	0.339	0.338 ถึง 0.341	0.438	0.70

สิ่งที่เห็น

- เกณฑ์ที่ต้องผ่านอยู่ที่ประมาณ 0.23 ถึง 0.40 แล้วแต่คลาสและกลุ่ม ใกล้กับที่ประมาณไว้ก่อนหน้านี้ (0.25 ถึง 0.30) แต่ Web Attack สูงกว่าที่ประมาณ

- ค่าที่ใกล้ที่สุดคือ DDoS กับตาราง SHAP ได้ 0.139 ต้องได้ 0.278 ประมาณครึ่งเดียว
- ค่า p จากการสุ่ม 200,000 ครั้งใกล้กับค่าจาก 2,000 ครั้งทุกค่า (เช่น DDoS ตาราง SHAP 0.159 กับ 0.144) ค่าตัดสินจึงไม่ได้ขึ้นกับจำนวนการสุ่ม
- ข้อสรุปของข้อ 13 และ 14 เหมือนเดิม

16. รอบ 3: RF, XGBoost, CatBoost

เกณฑ์: results/crossdataset/importance_comparison/SHAP_ROUND3_PREREGISTRATION.md เขียนก่อนรันสคริปต์ scripts/shap_round3.py ผลอยู่ที่ results/crossdataset/importance_comparison/shap_round3/ ผลรอบ 1 และรอบ 2 ยังเป็นค่าตัดสินเดิม

รอบ 2 ใช้แค่ LightGBM รอบนี้ถามว่าผลเดียวกันเกิดกับโมเดลต้นไม้ตัวอื่นใน protocol ด้วยหรือไม่

สิ่งที่เปลี่ยนจากรอบ 2

- ใช้ RF, XGBoost, CatBoost แทน LightGBM ตั้งค่าและเทรนเหมือน run_crossdataset.py
- CatBoost ใช้ SHAP ของ CatBoost เอง เพราะ shap.TreeExplainer ทำให้โปรแกรมพัง (access violation) ตอนอ่านโมเดล CatBoost
- ค่า SHAP ของ CatBoost รวมกันแล้วตรงกับผลของโมเดล ต่างไม่เกิน $2e-14$ (ทดสอบ 200 แถว)
- อย่างอื่นเหมือนรอบ 2 ทุกอย่าง: กลุ่ม F สามกลุ่ม, สุ่ม 2,000 ครั้ง seed 2026, Bonferroni
- class ที่ตัดสินได้คิดแยกแต่ละโมเดล RF ตัดสินได้ 4 คลาส (มี Bot เพิ่ม) XGBoost และ CatBoost ได้ 3 คลาส

Part B (ใช้ตัดสิน)

ทิศหลัก 2017 ไป 2018 กลุ่ม consensus (13) ค่าในวงเล็บคือ p

โมเดล	DDoS	DoS	Bot	Web Attack	ผล
LightGBM (รอบ 2)	-0.021 (0.57)	+0.066 (0.32)	ไม่ตัดสิน	-0.012 (0.59)	ไม่สนับสนุน
XGBoost	-0.074 (0.82)	+0.097 (0.17)	ไม่ตัดสิน	-0.012 (0.54)	ไม่สนับสนุน
CatBoost	+0.087 (0.17)	+0.177 (0.06)	ไม่ตัดสิน	-0.114 (0.76)	ไม่สนับสนุน
RF	-0.188 (0.94)	-0.107 (0.81)	-0.345 (0.97)	-0.236 (0.79)	ไม่สนับสนุน

ผลของทุกกลุ่ม feature

โมเดล	consensus (13)	SHAP (26)	perm (18)
XGBoost	ไม่สนับสนุน	ไม่สนับสนุน	ไม่สนับสนุน
CatBoost	ไม่สนับสนุน	ไม่สนับสนุน (DDoS ไม่ชัด p = 0.021)	ไม่สนับสนุน (DoS ไม่ชัด p = 0.041)
RF	ไม่สนับสนุน	ไม่สนับสนุน	ไม่สนับสนุน

"ไม่ชัด" คือ p ต่ำกว่า 0.05 แต่ไม่ถึงเกณฑ์ Bonferroni ($0.05 / 3 = 0.0167$)

ค่าตัดสินรอบ 3: ไม่สนับสนุน (3 จาก 3 โมเดล เกณฑ์ต้องการ 2 จาก 3)

Part A (รายงานอย่างเดียว ไม่ตัดสิน)

ยังไม่มีผล permutation ของ 3 โมเดลนี้ จึงเทียบบทบาทของ feature กับตาราง SHAP ของ LightGBM แทน

โมเดล	kappa เทียบ LightGBM	Spearman 2017	Spearman 2018	feature ที่ใช้แยกการโจมตี	บทบาทต่างจาก LightGBM
XGBoost	0.874	0.897	0.912	15	5 ตัว
CatBoost	0.770	0.866	0.857	15	9 ตัว
RF	0.626	0.699	0.821	28	16 ตัว

LightGBM ใช้ 18 feature ในการแยกการโจมตี

สิ่งที่เห็น

- ทั้ง 3 โมเดลได้ผลเหมือน LightGBM ไม่มีโมเดลไหนที่ feature fingerprint ผลักการโจมตีไปทาง Benign มากกว่ากลุ่ม feature สุ่มขนาดเท่ากัน
- DoS มีค่า G เป็นบวกในโมเดล boosting ทั้งสามตัว (สูงสุด CatBoost +0.177 $p = 0.06$) แต่ไม่มีค่าไหนผ่านเกณฑ์ และ RF เป็นลบทุกคลาส
- Part A: XGBoost และ CatBoost ใช้ feature ชุดใกล้กับ LightGBM มาก ส่วน RF กระจายน้ำหนักไปหลาย feature กว่า (28 ตัว) ตรงกับที่คาดไว้
- kappa ของ XGBoost และ CatBoost (0.77 ถึง 0.87) สูงกว่าที่คาดไว้ก่อนรัน (0.5 ถึง 0.7)
- ทิศ 2018 ไป 2017 (ไม่ใช่ดัดสิน) มี $p < 0.05$ แค่ 2 จาก 54 ค่า (ทั้งคู่เป็น XGBoost) ใกล้กับที่ความบังเอิญให้ได้ (ประมาณ 3 ค่า)
- ค่า p ต่ำของ LightGBM ในทิศ 2018 ไป 2017 (DDoS กับตาราง SHAP $p = 0.0015$ ในข้อ 13) ไม่เกิดซ้ำใน RF และ CatBoost ส่วน XGBoost ได้ $p = 0.047$

ปัญหาระหว่างรัน

- ครั้งแรก RAM ไม่พอตอน RF เริ่มเทรน ระบบหยุดงาน ผล XGBoost และ CatBoost บันทึกลงไฟล์ไว้แล้วจึงไม่หาย
- ครั้งที่สอง RF Part A เสร็จ แต่โดนหยุดตอนเริ่ม Part B จึงแก้สคริปต์ให้สร้างสรุป Part A จากไฟล์ part_a_roles.csv ได้
- ตรวจสอบว่าการสร้างสรุปจากไฟล์ให้ค่าของ XGBoost และ CatBoost ตรงกับการรันจริงทุกค่า
- แก้สคริปต์ให้ Part B บันทึกผลทุก seed ด้วย ถ้าโดนหยุดอีกจะรันต่อจาก seed ที่ค้างได้
- ครั้งที่สาม ความพยายามแรก RAM ไม่พอตอนโหลดข้อมูล 2018 สคริปต์ลองใหม่เองแล้วผ่าน
- จำนวนแถวแต่ละคลาสของข้อมูล 2018 ตรงกันทุกครั้งที่โหลด ทั้งในรอบ 2 และรอบ 3
- RF ช้าที่สุด ทิศ 2018 ไป 2017 ใช้ประมาณ 58 นาทีต่อ seed เพราะต้นไม้ที่เทรนจาก 2018 ใหญ่

ข้อจำกัด

- Part A ไม่มี permutation ของ 3 โมเดลนี้ จึงเป็นแค่การเทียบกับ LightGBM ไม่ใช่การยืนยันตาราง
- CatBoost ใช้ SHAP คนละตัวกับโมเดลอื่น (ค่าเป็น tree SHAP แบบเดียวกัน แต่เป็นโค้ดของ CatBoost)
- ยังไม่ได้ทำ MLP, LR และ stacking ใน Part B

17. สรุปหลังรอบ 3

1. โมเดลต้นไม้ทั้ง 4 ตัว (LightGBM, XGBoost, CatBoost, RF) ไม่มีหลักฐานว่า feature ที่บอก dataset เป็นตัวผลักการโจมตีให้ถูกทายเป็น Benign หลังย้าย dataset
2. ผลนี้ตรงกับ ablation เดิมที่ตัด feature fingerprint ออกแล้วไม่ดีขึ้น การตัด feature อย่างเดียวจึงไม่น่าจะแก้ปัญหาได้

3. คำอธิบายที่เหลือคือค่าของ feature เลื่อนไปทั้งหมด ขั้นตอนต่อไปจึงเป็น binary mode และ distribution alignment

4. ไม่วางแผนทำ SHAP รอบต่อไป ค่า p ต่ำในทิศ 2018 ไป 2017 เก็บไว้เป็นข้อสังเกตเท่านั้น